

Guía Práctica 6: DNS

Tomás F. Melli

November 2025

Índice

1	Ejercicio 1	2
2	Ejercicio 2	2
3	Ejercicio 3	4
4	Ejercicio 4	5
5	Ejercicio 5	5
6	Ejercicio 6	6
7	Ejercicio 7	7
8	Ejercicio 8	7
9	Ejercicio 9	8
10	Ejercicio 10	10
11	Ejercicio 11	10
12	Ejercicio 12	11

1 Ejercicio 1

La facultad decide instalar un servicio web al cual se espera que se conecten hasta 5 clientes simultáneamente.

1. ¿Cuántos servidores web son necesarios?
Uno, 5 personas es nada en términos de la carga que implican 5 conexiones concurrentes.
2. ¿Cuántas direcciones IP hacen falta?
Una para el servidor. Ya que a través de él podrán ingresar a la web. (Cada cliente tiene una IP)
3. ¿En cuántos puertos diferentes deben estar siendo atendidos?
El servidor “escucha” en **un** puerto (por ejemplo 80 (HTTP) o 443 (HTTPS)). Muchos clientes establecen conexiones TCP desde sus puertos efímeros hacia ese mismo puerto servidor: el servidor distingue cada conexión por la tupla (IP cliente, puerto cliente, IP servidor, puerto servidor).
4. ¿Y sabiendo que la cantidad de clientes va a aumentar?
Si crece la cantidad de clientes, tendremos que aumentar la cantidad de servidores. Del mismo modo, tendremos que encontrar maneras de distribuir el tráfico de manera adecuada para no saturar ninguno.

2 Ejercicio 2

dig (Domain Information Groper) es un comando que permite realizar consultas DNS.

Si miramos `man dig`:

```
1 DESCRIPTION
2     dig  is a flexible tool for interrogating DNS name servers.
3     It performs DNS lookups and displays the answers that are returned from the name server(s)
      that were queried.
4     Most DNS administrators use dig to troubleshoot DNS problems because of its flexibility, ease of
      use, and clarity of output. Other lookup tools tend to have less functionality than dig.
```

Con esto en mente, el uso será:

```
1 dig <nombre> <tipo_de_registro>
```

La salida tiene la forma:

```
1 ; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> www.dc.uba.ar
2 ;; global options: +cmd
3 ;; Got answer:
4 ;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41791
5 ;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1
6
7 ;; OPT PSEUDOSECTION:
8 ; EDNS: version: 0, flags:; udp: 65494
9 ;; QUESTION SECTION:
10 ;www.dc.uba.ar.          IN      A
11
12 ;; ANSWER SECTION:
13 www.dc.uba.ar.          394     IN      CNAME   www-1.dc.uba.ar.
14 www-1.dc.uba.ar.        394     IN      CNAME   dc.uba.ar.
15 dc.uba.ar.              394     IN      A       157.92.27.128
16
17 ;; Query time: 0 msec
18 ;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
19 ;; WHEN: Sat Nov 08 19:08:14 -03 2025
20 ;; MSG SIZE rcvd: 92
```

- **QUESTION SECTION:** Lo que dig preguntó (nombre + tipo de registro). Sirve para confirmar qué pediste.
- **ANSWER SECTION:** La respuesta principal del DNS.
- **AUTHORITY SECTION:** Indica qué servidores DNS son **autoritativos** para ese dominio. Solo útil para depurar o analizar jerarquía DNS.
- **ADDITIONAL SECTION:** Contiene datos adicionales útiles (por ejemplo, las IPs de los NS o de los MX).

1. Utilízelo para consultar registros del tipo A, CNAME y MX para los siguientes dominios:

- `www.dc.uba.ar`

– `dig www.dc.uba.ar A`: ¿Cuál es la dirección IPv4 asociada al nombre `www.dc.uba.ar`? La `A` es de Address.

```
1 ;; ANSWER SECTION:
2 www.dc.uba.ar.      600    IN      CNAME   www-1.dc.uba.ar.
3 www-1.dc.uba.ar.    600    IN      CNAME   dc.uba.ar.
4 dc.uba.ar.          600    IN      A       157.92.27.128
```

– `dig www.dc.uba.ar CNAME`: ¿`www.dc.uba.ar` es un alias (nombre alternativo) de otro dominio? El `CNAME` es de Canonical Name.

```
1 ;; ANSWER SECTION:
2 www.dc.uba.ar.      593    IN      CNAME   www-1.dc.uba.ar.
```

– `dig www.dc.uba.ar MX`: ¿Qué servidores de correo manejan el dominio `dc.uba.ar`? El `MX` es de Mail Exchange.

```
1 ;; ANSWER SECTION:
2 www.dc.uba.ar.      492    IN      CNAME   www-1.dc.uba.ar.
3 www-1.dc.uba.ar.    492    IN      CNAME   dc.uba.ar.
4 dc.uba.ar.          600    IN      MX       10 ALT3.ASPMX.L.GOOGLE.COM.
5 dc.uba.ar.          600    IN      MX       5 ALT2.ASPMX.L.GOOGLE.COM.
6 dc.uba.ar.          600    IN      MX       1 ASPMX.L.GOOGLE.COM.
7 dc.uba.ar.          600    IN      MX       10 ALT4.ASPMX.L.GOOGLE.COM.
8 dc.uba.ar.          600    IN      MX       5 ALT1.ASPMX.L.GOOGLE.COM.
```

- `dc.uba.ar`
- `uba.ar`
- `ar`

2. Idem con la opción `+trace`.

El parámetro `+trace` en `dig` activa un modo de resolución recursiva paso a paso, donde vemos toda la cadena de consultas DNS desde la raíz hasta el dominio final. O sea, `dig +trace` muestra cómo se resuelve un nombre de dominio siguiendo el mismo proceso que haría un resolver DNS, pero sin usar caché ni servidores intermedios.

- Consulta a un servidor raíz (`.`) para saber qué servidores son responsables del dominio de nivel superior `.ar`.
→ Aparecen registros `NS` del TLD `.ar`.
- Consulta a uno de los servidores del TLD `.ar` para averiguar quién es responsable del subdominio `uba.ar`.
→ Recibe nuevos registros `NS` para `uba.ar`.
- Consulta a los servidores de `uba.ar` para obtener los registros del subdominio `dc.uba.ar`.
→ Recibe la respuesta final (por ejemplo, un registro `A` con la IP).

3. ¿Cómo se reconocen los servidores autoritativos de una determinada zona?

Un servidor DNS es **autoritativo** para una zona si contiene los registros originales de esa zona.

- **Respuesta DNS**: Si el servidor es autoritativo, el campo `AA` (**Authoritative Answer**) de la respuesta DNS estará activado. Con `dig`, se observa esto en el campo `flags` de la respuesta:

```
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
```

- **Comando `dig` con registros `NS`**: Se puede consultar los servidores `NS` (**Name Server**) para un dominio, ya que estos son los servidores autoritativos para esa zona: `dig dc.uba.ar NS`
- **Registro `SOA`**: El registro `SOA` (**Start of Authority**) de una zona contiene información sobre el servidor primario de la zona y permite identificar a los servidores autoritativos. Para realizar la consulta del registro `SOA` hacemos: `dig dc.uba.ar SOA`

4. ¿Cómo se distinguen los servidores secundarios del servidor primario?

Los **servidores secundarios** son réplicas del servidor primario y obtienen la información mediante un proceso de **transferencia de zona** (`zone transfer`).

- **Registro `SOA`**: El registro `SOA` (**Start of Authority**) incluye el nombre del servidor primario en el campo `MNAME`, lo que permite identificarlo.
- **Diferencias operativas**:
 - **Servidor primario**: Almacena los registros originales y puede ser modificado directamente por el administrador.

- **Servidores secundarios:** Solo pueden realizar transferencias de zona desde el primario. Esto se puede observar en herramientas de monitoreo DNS (por ejemplo, con Wireshark se pueden capturar transferencias de zona).
- **Wireshark:** Usando **Wireshark**, podemos filtrar los mensajes DNS para ver interacciones como transferencias de zona (**AXFR** o **IXFR**), donde los servidores secundarios solicitan información al primario.
- Al hacer una consulta DNS para **www.ejemplo.com**, es común que el nombre esté configurado como un alias mediante registros **CNAME**.
- En este caso, el servidor DNS puede devolver varios **CNAME** antes de obtener la dirección IP final. Por ejemplo:
 - **www.ejemplo.com** → **CNAME** → **alias1.ejemplo.com** → **CNAME** → **alias2.ejemplo.com** → **A** → **192.0.2.1**
- Esto significa que **www.ejemplo.com** no tiene una IP propia, sino que apunta a otro dominio que finalmente resuelve en una dirección IP.
- En cambio, al consultar el dominio sin **www** (por ejemplo, **ejemplo.com**), normalmente se obtiene directamente un registro **A** o **AAAA**, sin pasos intermedios:
 - **ejemplo.com** → **A** → **192.0.2.1**
- Si se ejecuta la consulta con el parámetro **+trace**, se obtiene el seguimiento completo del proceso de resolución DNS:
 - Desde los servidores raíz (**.**)
 - Luego los del TLD (por ejemplo, **.com** o **.ar**)
 - Hasta llegar a los servidores autoritativos del dominio consultado.

Ayuda: Buscar la especificación del registro **SOA** en el RFC 1035.

Otra ayuda: Analizar los mensajes DNS que se envían usando **Wireshark**.

Nota: Para quienes no cuentan con **dig** en su SO, pueden utilizar www.digwebinterface.com.

3 Ejercicio 3

El protocolo HTTP permite hacer distintos tipos de pedidos para recursos de un determinado dominio. Escriba los Requests HTTP 1.1 que permitan obtener los siguientes pedidos al sitio web del departamento de computación:

El recurso /

Para obtener el recurso / :

```
1 GET / HTTP/1.1
2 Host: www.dc.uba.ar
```

Encabezado del recurso /tdc

```
1 HEAD /tdc HTTP/1.1
2 Host: www.dc.uba.ar
```

El recurso /logo.jpg si no fue modificado desde una determinada fecha

```
1 GET /logo.jpg HTTP/1.1
2 Host: www.dc.uba.ar
3 If-Modified-Since
```

Crear un nuevo post en el foro del departamento

```
1 POST /foro HTTP/1.1
2 Host: www.dc.uba.ar
3 Content-Type: application/json
4 Content-Length: 70
5 <datos>
```

4 Ejercicio 4

Un usuario se sienta en su computadora hogareña. Consulta el estado del tiempo en la página del servicio meteorológico nacional (www.smn.gov.ar) y luego manda un mail usando un user agent (por ejemplo, Thunderbird) a una dirección en el dominio dc.uba.ar.

¿Cuántas conexiones TCP desencadena el usuario?

- El usuario accede por HTTP al sitio web www.smn.gov.ar. Este acceso dispara una solicitud HTTP GET que utiliza una conexión TCP (suponiendo HTTP/1.1 y que todos los recursos solicitados están en el mismo servidor).
- Luego, el usuario envía un correo usando SMTP a una dirección del dominio dc.uba.ar.
 - El cliente de correo (en la computadora del usuario) establece una conexión TCP con el servidor de correo saliente (SMTP), generalmente al puerto 587 o 465 (SMTP autenticado).
 - El servidor de correo saliente establece otra conexión TCP con el servidor de correo entrante del dominio de destino (dc.uba.ar) para finalizar el envío del correo. Esta conexión se realiza típicamente al puerto 25 (SMTP sin autenticación entre servidores).
- En total, se establecen **3 conexiones TCP**.

¿Qué computadoras se comunican usando UDP? Describa una posible secuencia de mensajes DNS que se desencadenaría para concretar el envío del mail. *Asuma que las caches están vacías (DNS y HTTP).*

- Dado que las cachés están vacías, todas las consultas deben enviarse al *resolver DNS local* del usuario y/o del servidor de correo.
- **Primera consulta:** Desde la computadora del usuario se consulta a su resolver local por la dirección IP de www.smn.gov.ar.
 - Si el resolver no tiene la respuesta, desencadena una cadena de consultas:
 - * al servidor raíz (Root DNS) por .ar,
 - * al servidor TLD .ar por smn.gov.ar,
 - * y finalmente al servidor autoritativo para smn.gov.ar, que devuelve la dirección IP.
- **Segunda consulta:** Desde la computadora del usuario también a su resolver local, se consulta la dirección IP del servidor de correo saliente (smtp.miisp.com, por ejemplo), necesaria para establecer la conexión SMTP.
- **Tercera consulta:** El servidor de correo saliente realiza una consulta DNS a su resolver local por el registro MX del dominio dc.uba.ar, para conocer el servidor que recibe el correo.
 - Esta consulta sigue una cadena de resoluciones similar:
 - * el resolver pregunta al servidor raíz por .ar,
 - * luego al TLD .ar por uba.ar,
 - * y finalmente al servidor autoritativo de dc.uba.ar, que devuelve el registro MX.
- Ejemplo de mensaje DNS: `request dc.uba.ar MX`.

5 Ejercicio 5

En algunos casos un `request` HTTP dispara además una consulta DNS. En términos de performance, las decisiones de diseño de DNS afectan también a HTTP.

Pensar las implicancias para HTTP si DNS es centralizado (en contraposición a distribuido), o si no implementa caché, o si funciona orientado a conexiones. ¿Se puede usar el mismo razonamiento con SMTP?

Si DNS fuera centralizado, todos los requests deberían atenderse en un mismo servidor. Todas las máquinas deberían buscar direcciones en el mismo lugar, generando una saturación en el server ya que tiene que atender todas las request que recibe. Se tarda una cantidad inmensa de tiempo en buscar una dirección, ya que hay que iterar sobre todas las direcciones del server. La falta de redundancia hace que el sistema sea menos resiliente a fallos o ataques (por ejemplo, ataques DDoS). Para el envío de correos con SMTP, los servidores de correo necesitan resolver los registros MX de los dominios destino. Si DNS es centralizado, un alto volumen de tráfico en el sistema puede ralentizar significativamente el proceso de entrega de correos.

Si DNS no tuviera un sistema de cacheo, cada request debería ser siempre resuelta en su totalidad. Si yo ingreso a dc.uba.ar, cierro la página y vuelvo a ingresar, tengo que esperar a que el Resolver haga la consulta DNS en su totalidad, en vez de devolver la IP al instante porque la tiene cacheada. Esto tiene cierto impacto en la performance de HTTP, aunque sean algunos milisegundos. SMTP: Sin cacheo, los servidores de correo tendrían que realizar consultas DNS repetitivas para cada correo enviado, incluso si los correos son para el mismo dominio. Esto aumenta la latencia en la entrega de correos.

Si DNS fuera orientado a conexiones (TCP) habría una latency más grande con cada consulta. Habría que establecer una conexión TCP nueva con cada consulta con el 3-way handshake. Además, las respuestas DNS serían mucho más grandes y pesadas, ya que TCP tiene un header más grande que UDP. El resolver abriría una nueva conexión mientras resuelve la consulta DNS iterativamente. HTTP sería muy lento. SMTP también se vería afectado, ya que las consultas DNS para registros MX y A (o AAAA) tendrían mayor latencia. Dado que SMTP depende de múltiples consultas DNS para entregar un correo, el tiempo total de entrega se incrementaría significativamente.

En conclusión se puede aplicar el mismo razonamiento para SMTP. La latencia en HTTP se refleja en que la página tarda en abrir, y en SMTP en que los mails tardan en llegar y se encolan en la cola de salida.

6 Ejercicio 6

Suponiendo que tenemos una página web en HTML que reside en el servidor `www.fcen.uba.ar` como la que aparece a continuación:

```
<html>
  <head>
    <title>Facultad de Ciencias Exactas y Naturales</title>
    <link rel="stylesheet" type="text/css" href="style.css" />
  </head>
  <body>
    <div>
      
      <a href="avsearch.php">  </a>
    </div>
    <div>
      <form name="searchform" action="search">
        <label>Buscar</label>
        <input name="SearchableText" type="text" title="Buscar en el Sitio" />
        <input type="image" src="search_icon.gif" />
      </form>
    </div>
  </body>
</html>
```

Asumir que las ventanas de las conexiones (RWND y CWND) son lo suficientemente grandes como para que cada archivo no necesite más de 1 RTT para enviarse.

¿Cuánto tiempo en términos de RTTs transcurrirá como mínimo, hasta transferir la totalidad de la información en HTTP/1.0?

Como es 1.0 entonces va a haber una conexión nueva para cada objeto que se deba traer del servidor y tomamos cada una como un RTT. Entonces tenemos :

- $2 \times RTT$ por el envío del documento HTML propiamente dicho.
- $2 \times RTT$ para traer la hoja de estilos CSS.
- $2 \times RTT$ para traer la imagen `searchline.png`.
- $2 \times RTT$ para traer la imagen `home.png`.
- $2 \times RTT$ para traer el ícono `searchicon.gif`.

avsearch.php no se trae inicialmente, se procesa su pedido cuando el cliente clickee sobre el link.

¿Y en HTTP/1.1?

Comparativa de tiempos con y sin *pipelining*

- En el caso de utilizar *pipelining*:
 - $1 \times RTT$ para inicializar la conexión.

- $1 \times RTT$ para traer el documento **HTML**.
- $1 \times RTT$ para traer todas las imágenes y hojas de estilo **CSS**.

Esto sucede así porque, al no cerrarse la conexión para cada pedido, se puede asumir que al usar *pipelining* la transferencia de los archivos incluidos en el HTML ocurre dentro de un único ciclo de *RTT*.

• **En el caso de no utilizar *pipelining*:**

- $1 \times RTT$ para inicializar la conexión.
- $1 \times RTT$ para traer el documento **HTML**.
- $1 \times RTT$ por cada imagen y hoja de estilo **CSS**.

7 Ejercicio 7

A continuación se enumeran dos bases de datos de registros DNS de dos name servers autoritativos. Explicar la secuencia de consultas DNS para encontrar primero la dirección IP de un servidor de mails para el dominio **uba.ar** y luego la del nombre de dominio **milagros.dc.uba.ar**, explicita los registros que se cachean.

Suponer que ya está cacheada la dirección del name server de uba.ar.

uba.ar.	1w	IN	SOA	uba.ar backup.servidormisterioso.ar admin.uba.ar(2005091900 ; serial 3h ; refresh 1h; retry 1w; expire 1h); neg cache
uba.ar.		IN	NS	servidores.uba.ar
uba.ar.		IN	MX	mailserver.uba.ar
rectorado		IN	CNAME	secretaria.uba.ar
dc.uba.ar.		IN	NS	servidores.dc.uba.ar
servidores		IN	A	208.25.19.1
servidores.dc.uba.ar.		IN	A	208.190.1.4
mailserver		IN	A	208.25.19.2
secretaria		IN	A	208.25.19.87
dc.uba.ar.	1w	IN	SOA	dc.uba.ar mateo.dc.uba.ar(2005091900 ; serial 3h ; refresh 1h; retry 1w; expire 1h); neg cache
dc.uba.ar.		IN	NS	servidores.dc.uba.ar
dc.uba.ar.		IN	MX	mailserver.dc.uba.ar
servidores.dc.uba.ar.		IN	A	208.190.1.4
mailserver		IN	A	208.190.1.32
milagros		IN	A	208.190.1.15
morza		IN	A	208.190.1.20

8 Ejercicio 8

La incorporación del formato HTML en correo electrónico permitió enviar texto con formato e imágenes embebidas. Explique cómo se puede lograr el envío y procesamiento de un nuevo formato de archivo (por ejemplo, la transición de texto plano a HTML) sin que esto implique un cambio en todos los servidores SMTP del mundo.

Pista: ver MIME.

MIME (Multipurpose Internet Mail Extensions) es una extensión del estándar **RFC 822**, que define el formato básico de los mensajes de correo electrónico. MIME amplía este formato para permitir que los mensajes contengan distintos tipos de contenido, como texto en diferentes codificaciones, imágenes, audio, video o archivos adjuntos

- El mensaje comienza con un **header**, que incluye información sobre la estructura del cuerpo (*body*) del mensaje.
- El **body** puede contener múltiples partes separadas por un **boundary**, una cadena especial que actúa como delimitador entre secciones.
- Cada parte del mensaje está precedida por una línea **Content-Type**, donde se especifica el tipo de contenido (por ejemplo, texto, HTML, imagen, etc.).

- Gracias a esto, el cliente de correo sabe cómo interpretar y mostrar cada parte del mensaje.

Un ejemplo de mensaje MIME sería:

```
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="boundary_string"
```

```
--boundary_string
Content-Type: text/plain; charset="utf-8"
```

Este es el contenido en texto plano.

```
--boundary_string
Content-Type: text/html; charset="utf-8"
```

```
<html>
<body>
<p>Este es el contenido en <b>HTML</b>.</p>
</body>
</html>
```

```
--boundary_string
Content-Type: image/jpeg
Content-Disposition: attachment; filename="imagen.jpg"
Content-Transfer-Encoding: base64
```

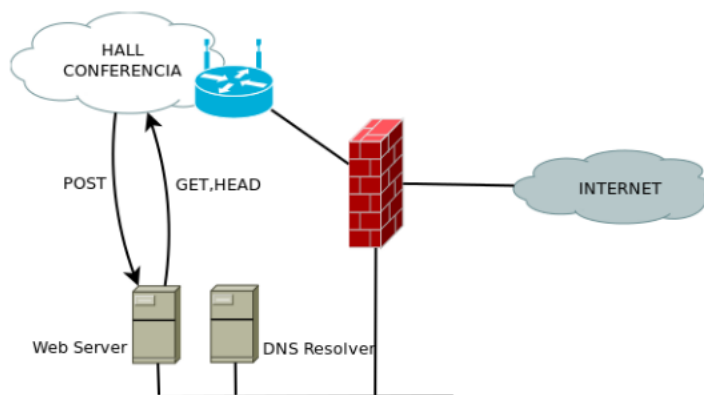
```
/9j/4AAQSkZJRgABAQEAAAAAAAAAD...
```

```
--boundary_string--
```

En este ejemplo se observa cómo cada parte del *body* está delimitada por el **boundary**, y cómo el campo **Content-Type** indica el tipo de contenido que sigue.

9 Ejercicio 9

En una conferencia se organiza un sistema donde un dispositivo hace un **POST** al servidor web enviando contenidos y especificando el tipo de recurso. Otros dispositivos acceden al recurso con **GET** o **HEAD**.



Usando MIME, describa posibles Requests **POST**, **GET** y **HEAD** involucrados entre los asistentes y el Web Server.

Los métodos **HTTP** definen las operaciones que un cliente puede realizar sobre un recurso en un servidor web ([RFC 7231](#)).

Método POST

El método POST se utiliza para enviar datos al servidor, normalmente como parte de un formulario o para subir archivos. Los datos se incluyen en el cuerpo del mensaje, pudiendo adoptar distintos formatos como `application/x-www-form-urlencoded` o `multipart/form-data`.

```
POST /upload HTTP/1.1
Host: conferenciaserver.com
Content-Type: multipart/form-data; boundary=---boundary123

---boundary123
Content-Disposition: form-data; name="file"; filename="paper.pdf"
Content-Type: application/pdf

<contenido binario del archivo PDF aquí>

---boundary123{
```

El servidor procesa esta solicitud, guarda el archivo en el sistema y devuelve una respuesta HTTP. Por ejemplo:

```
HTTP/1.1 200 OK
Content-Type: text/plain
```

Archivo recibido correctamente.

En caso de error, puede devolver un código como 400 Bad Request o 500 Internal Server Error.

Método GET

El método GET solicita al servidor la recuperación de un recurso identificado por una URL. Los parámetros, si los hay, se envían en la propia línea de la petición (no en el cuerpo).

```
GET /uploads/paper.pdf HTTP/1.1
Host: conferenciaserver.com

El servidor responde con el contenido solicitado:

HTTP/1.1 200 OK
Content-Type: application/pdf
Content-Disposition: attachment; filename="paper.pdf"
Content-Length: 204800
```

<contenido del archivo PDF>

Método HEAD

El método HEAD es idéntico a GET, salvo que el servidor responde solo con las cabeceras, sin incluir el cuerpo del mensaje. Se utiliza para obtener metadatos del recurso (por ejemplo, su tipo o tamaño) sin descargarlo.

```
HEAD /uploads/paper.pdf HTTP/1.1
Host: conferenciaserver.com
```

Respuesta del servidor:

```
HTTP/1.1 200 OK
Content-Type: application/pdf
Content-Length: 204800
```

Esto permite verificar la existencia o el tamaño de un recurso sin transferir su contenido completo.

10 Ejercicio 10

Una empresa de spam envía mails usando un host `spameitor.unaempresadespam.com.ar`. El servicio tiene configurado como servidor saliente a `smtp.unaempresadespam.com.ar`. El envío consiste en usar el siguiente cuerpo (se reemplaza `[[Mail]]` por cada dirección destinataria):

```
To: [[Mail]]
From: "UnaEmpresaDeSpam.com.ar" <ofertas@unaempresadespam.com.ar>
Reply-to: "UnaEmpresaDeSpam.com.ar" <ofertas@unaempresadespam.com.ar>
Subject: En Esta Navidad Regala Ahorrando!
MIME-Version: 1.0
Content-Type: text/html; charset = "iso-8859-1"
```

```
<html> <head></head>
<body><div>
  Feliz Navidad! Compre!! OFERTAS, OFERTAS, BLABLABLA!!! <br />
  <br />
  <a href="http://envios.unaempresadespam.com.ar/unsubscribe.php?CampaignID=10057&Email=[[Mail]]">
    Si no queres recibir mas nuestros mails hace click aca para desuscribirte
  </a>
</div>

</body>
</html>
```

Base de Mails:

```
bob@uba.ar
alice@uba.ar
trudy@uba.ar
bob@dc.uba.ar
alice@dm.uba.ar
trudy@df.uba.ar
```

1. Describa todas las consultas DNS necesarias para poder enviar los mails, asumiendo que los hosts mencionados usan como resolver a `dns-1.unaempresadespam.com.ar` y que las consultas a ese resolver casi siempre están en caché. El resto de las cachés están vacías.
2. Describa los mensajes HTTP (Requests y Responses) necesarios para poder mostrar el mail (ev. recursos remotos referenciados en el HTML).

11 Ejercicio 11

Juancho quiere acceder a su cuenta de webmail en `www.jmail.com` desde un navegador. Su PC conoce la dirección de su resolver DNS. Suponiendo que todos los caches involucrados están vacíos al inicio y que no expiran durante la sesión:

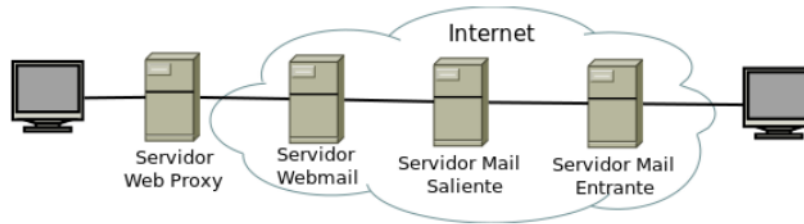
```
jmail.com      1w IN    SOA jmail.com

jmail.com.      IN      NS      admin.jmail.com (
jmail.com.      IN      MX      2005091900 3h 1h 1w 1h)
mailserver      IN      MX      ns.jmail.com
cdn.akamai.com  IN      CNAME   mailserver.jmail.com
www              IN      CNAME   elmailserver.jmail.com
ads              IN      CNAME   recursos.jmail.com
elmailserver     IN      A        208.190.1.20
ns                IN      A        208.190.1.21
server1          IN      A        208.190.1.22
recursos         IN      A        208.190.1.26
<html>
  <head>
    <title> jmail.com </title>
  </head>
  <body>
    <div>
      
    </div>
    <div>
      <label> Bandeja de entrada </label>
      <table style="width:100%">
        <tr>
          <td> Entrega tp1 - wiretapping </td>
        </tr>
        <tr>
          <td> Entrega tp2 - rutas </td>
        </tr>
      </table>
      <button type="button"> Ver 3 emails más </button>
    </div>
    <button type="button"> Redactar nuevo jmail </button>
  </div>
  <iframe src="http://ads.spameitor.com"></iframe>
</div>
</body>
</html>
```

- Detalle los mensajes HTTP (Requests/Responses) y los mensajes DNS (Consultas/Respuestas) que se desencadenan al ingresar a `www.jmail.com`.
- ¿Cuántas conexiones TCP genera?

12 Ejercicio 12

Se muestra un flujo: desde una computadora se accede a un Webmail usando un proxy HTTPS. El servidor de Webmail envía el correo usando un servidor SMTP; el mail llega al servidor de mail del dominio de destino, y ese correo es descargado por otra computadora usando POP3.



1. Describa todas las conexiones TCP que se establecen durante todo el proceso y qué protocolos de la capa de aplicación transportan.
2. ¿En qué momento se realiza la consulta DNS por el registro MX del dominio de destino?